Pac-Man: Data and Architecture Design

Max Haijes (8073201), Anass Zaoujal (2316277)
October 14, 2026

1 Introduction and Rules

This document describes the data and architecture design for our Pac-Man style game. Pac-Man is a game where the goal is to navigate a maze, collecting pellets to clear the board while avoiding (at least) four antagonistic ghosts. The player controls their character with the arrow keys, allowing up, down, left and right movement. Every ghost has their own movement strategy to try and catch you. If the player touches a ghost without a powerup they lose a life. When the player runs out of lives, the game is over. There are also energizers across the map which temporarily frighten the ghosts allowing the player to eat them for bonus points.

2 Design

2.1 Data Structures

2.1.1 The Game State

The entire state of the game is stored in a single GameState record which is used across our model-view-controller setup. It keeps track of the player, the ghosts, the current maze layout and general game info like the score, pause state and a random number generator for mechanics like frightened ghost movement.

```haskell
import System.Random (StdGen)

data GameState = GameState
  { player         :: PacMan
  , ghosts         :: [Ghost]
  , grid           :: Grid
  , score          :: Int
  , lives          :: Int
  , currentMode    :: GhostMode
  , modeTimer      :: Float
  , isPaused       :: Bool
  , rng            :: StdGen
  , animationTimer :: Float
```

```
  , deathAnimation :: Maybe Float
  , highScore      :: Int
  , currentLevel   :: Int
  , gameScreen     :: GameScreen
  } deriving (Show)
```

2.1.2 The Player and the Playing Field

The playing field is a 2D grid made up of different tile types. We represent grid coordinates using a simple (Int, Int) tuple. The PacMan record tracks the player's exact location, their current moving direction and a "queued" direction to allow for smooth cornering (input buffering). Gate is the tile where ghosts exit their 'house'.

```
type Position = (Int, Int)

data Tile
  = Wall
  | Pellet
  | Energizer
  | Empty
  | Gate
  deriving (Eq, Show)

type Grid = [[Tile]]

data PacMan = PacMan
  { pacmanPos   :: Position
  , pacmanDir   :: Direction
  , queuedDir   :: Maybe Direction
  } deriving (Show)
```

2.1.3 Player Movement

The movement in Pac-Man is restricted to four discrete directions. Player input does not move the character instantly. Instead, it buffers an intended turn. When the character reaches the next valid tile intersection, the game evaluates if the queued direction is clear of Wall tiles. We do this to ensure smooth movement and restrict unwanted collision errors.

```
data Direction
  = Up
  | Down
  | Left
  | Right
  deriving (Eq, Show)
```

```
bufferInput :: Event -> PacMan -> PacMan

updatePacManPosition :: Grid -> PacMan -> PacMan
```

2.1.4 Level Data types

In order for our implementation of multiple levels to work, we are adding a Level data type. This allows each level to have its own maze layout, ghost movement speed and allows us to add more difficult ghosts depending on the maze level. We also use a GameScreen data type to keep track of which screen is currently displayed.

```
data GameScreen
  = StartScreen
  | LevelSelect
  | Playing
  | GameOver
  deriving (Eq, Show)

data Level = Level
  { levelNumber :: Int
  , levelGrid   :: Grid
  , ghostSpeed  :: Float
  , ghostRoster :: [GhostType]
  } deriving (Show)
```

The currentLevel and gameScreen fields in GameState will keep track of the current level and the current screen. The selected level determines which maze, ghost types and movement speed are used during gameplay.

2.2 Computer Adversaries

Pac-Man does not feature a direct opponent but rather four enemies that move around the playing field with different predetermined behavioral patterns.

There is “Chase” mode which enables the basic behaviours described below (first to implement).

There is also “Scatter” mode and “Frightened” mode. During scatter mode, each ghost abandons the goals of chasing down Pac-Man and starts navigating towards and around a predetermined corner of the playing field (different one for each ghost). Frightened mode

means the ghosts turn blue, they slow down, move ‘randomly’ at intersections and Pac-Man is able to eat the ghosts for bonus points.

To represent the ghosts, we define separate data types for their colour and current mode, along with a Ghost record to hold each ghost’s state:

```
data GhostType
 = Red
 | Pink
 | Cyan
 | Orange
 | Purple
 deriving (Eq, Show)

data GhostMode
 = Chase
 | Scatter
 | Frightened
 deriving (Eq, Show)

data Ghost = Ghost
 { ghostType      :: GhostType
 , ghostPos       :: Position
 , ghostDir       :: Direction
 , ghostMode      :: GhostMode
 , homeCorner     :: Position
 , frightenedTime :: Float
 } deriving (Show)
```

At any intersection, a ghost chooses the valid non-reversing direction that minimizes the straight-line distance to its current target tile. In scatter mode, a ghost targets its assigned home corner. In chase mode, the target is computed based on its specific identity. In frightened mode, decisions are determined pseudo-randomly rather than through target minimization.

The ghostTarget function connects all the individual ghost behaviours using the current game state:

```
ghostTarget :: Ghost -> GameState -> Position
ghostTarget ghost gamestate = case ghostMode ghost of
  Scatter    -> homeCorner ghost
  Frightened -> ghostPos ghost
```

```
  Chase      -> case ghostType ghost of
    Red    -> targetRed (pacmanPos (player gamestate))
    Pink   -> targetPink (pacmanPos (player gamestate)) (pacmanDir
(player gamestate))
    Cyan   -> targetCyan (pacmanPos (player gamestate)) (pacmanDir
(player gamestate)) (getRedPos gamestate)
    Orange -> targetOrange (ghostPos ghost) (pacmanPos (player
gamestate)) (homeCorner ghost)
    Purple -> targetPurple ghost gamestate

getRedPos :: GameState -> Position
getRedPos gamestate =
  case filter (\g -> ghostType g == Red) (ghosts gamestate) of
    (red:_) -> ghostPos red
    []      -> pacmanPos (player gamestate)

chooseNextMove :: Position -> [Direction] -> Position -> Direction
chooseNextMove currentPos validDirs target =
  minimumBy (comparing (distance target . stepInDir currentPos))
validDirs
```

2.2.1 Red (Chaser)

The red ghost's primary behaviour consists merely of moving towards Pac-Man's exact current grid tile. This ghost will constantly be chasing the player down from behind to exert pressure.

```
targetRed :: Position -> Position
targetRed pacmanPos = pacmanPos
```

2.2.2 Pink (Interceptor)

The pink ghost's behaviour consists of targeting the tile that is four tiles ahead of Pac-Man in the direction that Pac-Man is currently facing. This means that pink's target changes as soon as Pac-Man moves, changes direction or both. The goal of this behaviour is to intercept Pac-Man and cut their escape route off at upcoming intersections rather than merely following Pac-Man around.

```
targetPink :: Position -> Direction -> Position
targetPink (x, y) dir = case dir of
  Up    -> (x, y + 4)
  Down  -> (x, y - 4)
  Left  -> (x - 4, y)
  Right -> (x + 4, y)
```

2.2.3 Cyan (Flanker)

Cyan's behaviour has a close relation to Red's behaviour. Cyan's target is the tile that is the result of the following operations: 1. Take the tile two tiles ahead of Pac-Man in his current direction. 2. Take the vector from Red's current position to the resulting tile from step 1. 3. Double that vector. The resulting tile is Cyan's target. This means that Cyan's behaviour is dependent on Pac-Man's inputs, similar to Pink.

```
targetCyan :: Position -> Direction -> Position -> Position
targetCyan pacmanPos pacmanDir redPos =
  let intermediate = targetAhead 2 pacmanPos pacmanDir
      vector = intermediate `subtractPos` redPos
  in intermediate `addPos` vector
```

2.2.4 Orange (Coward)

Orange's behaviour is dependent on how close Pac-Man is to Orange's current position. If Pac-Man is within eight tiles, Orange will aim to move to their specific home corner. If Pac-Man is outside of the eight tile range, Orange will behave the same as Red, targeting Pac-Man's current tile.

```
targetOrange :: Position -> Position -> Position -> Position
targetOrange orangePos pacmanPos orangeHomeCorner
  | distance orangePos pacmanPos > 8 = pacmanPos
  | otherwise                        = orangeHomeCorner
```

2.2.5 Purple (Polymorph)

Unlike the other ghosts, Purple does not follow one fixed strategy for movement. It instead dynamically chooses between the existing ghost strategies depending on the current game state. For example, when Pac-Man is close, Purple can turn into Red's chasing behaviour, while at intersections it may use Pink's interception strategy. Purple can also use Cyan's flanking strategy when Red is already close to Pac-man.

This allows Purple to adapt its behaviour to the different situations that can arise during gameplay, making it a much more difficult ghost to predict.

The targetPurple function will evaluate the current game state and determine which targeting strategy to use while its base behaviour is the same as that of the pink ghost.

We don't yet exactly know how we will be implementing purple's logic but this should give an idea of what it will look like.

```
targetPurple :: Ghost -> GameState -> Position
targetPurple ghost gamestate
  | pacmanIsNear gamestate = targetRed pPos
  | redIsClose gamestate   = targetCyan pPos pDir (getRedPos gamestate)
  | otherwise              = targetPink pPos pDir
  where
    pPos = pacmanPos (player gamestate)
    pDir = pacmanDir (player gamestate)

    pacmanIsNear :: GameState -> Bool
    pacmanIsNear = undefined

    redIsClose :: GameState -> Bool
    redIsClose = undefined
```

2.3 Interface

The game will use the Gloss library to display a graphical interface, following a Model-View-Controller (MVC) architecture. The model consists of the data structures described in Section 2.1.

The view is responsible for displaying the game using information stored in GameState. It will render the Pac-Man, ghosts, maze, pellets, the current score and the remaining lives. The Controller will handle keyboard input and update the game state accordingly. A separate key will allow the user to pause and resume the game.

The main functions for the interface will be:

```
initialGame :: IO GameState
renderGame  :: GameState -> Picture
handleInput :: Event -> GameState -> GameState
updateGame  :: Float -> GameState -> GameState
```

The renderGame function takes the current game state and turns it into a picture without modifying it. The handleInput function processes keyboard input events and uses bufferInput to store the player's intended direction. Finally, the updateGame function updates the game as time passes, including player movement, ghost movement and collision detection. These functions will remain pure and will be connected to Gloss through IO wrappers where necessary.

2.4 Implementation of Minimum Requirements

Player: the player controls Pac-Man using the arrow keys. The handleInput function processes keyboard events and uses bufferInput to store the intended direction. The updatePacManPosition function then updates Pac-Man's position, taking wall collision into account.

Enemies: there are four classic types of ghosts (plus a fifth polymorphic ghost introduced in later levels), each with their own movement strategy. The ghostTarget function determines each ghost's target based on its type and current mode. The ghosts will move automatically as the game state is updated.

Randomness: randomness will mainly be used for the movement of frightened ghosts. When a ghost becomes frightened, it will start to randomly choose between valid directions at intersections. We will use the StdGen stored in GameState to get these random values and update the generator after each use.

Animation: we plan to implement at least two animations for Pac-Man. The first is the eating animation, where Pac-Man's mouth continuously opens and closes as he moves along the maze. The second animation is a death animation that plays when Pac-Man collides with a ghost. These animations will be tracked using the animationTimer and deathAnimation fields in GameState, which are updated by updateGame.

Pause: the player can pause and resume the game by pressing a specific keyboard key. The isPaused field in GameState keeps track of whether the game is paused or not. When the isPaused Boolean is set to true, the game is paused. This means that updateGame will not update the position of Pac-Man or the ghosts, the game timers are also paused.

File I/O: the game will use the file system to save and load high scores. When the game is started, the saved high score will be read from a file. When the player finishes a game with a new high score, that new high score will be updated and written to a file. The high score will be stored in the highScore field of GameState, while reading and writing the high-score file will be handled through IO.

```
loadHighScore :: FilePath -> IO Int
saveHighScore :: FilePath -> Int -> IO ()
```

2.5 Implementation of Optional Requirements

Levels and Different Enemies: We plan to implement three different levels, each with its own maze and increasing difficulty. The player will be able to select one of the three levels from a level-selection menu before starting the game. As the player progresses through the levels, the ghosts will become faster, making it more difficult to avoid them. In the third level, we also plan to introduce a fifth ghost with a different appearance and movement strategy from

the original four ghosts. Unlike the original four ghosts, Purple will adapt its movement strategy depending on the current game state. It will choose between the existing ghost strategies based on factors such as Pac-Man's position and the positions of other ghosts. We will use a Level data type to store the different maze layouts, ghost speeds and available ghost types. The loadLevel function will initialize the game based on the selected level.

```
loadLevel :: Level -> GameState -> GameState
```

Saving and loading game state: we plan to allow players to save and load their game progress using JSON. We will use the Aeson library to serialize the current GameState into JSON and write it to a file. This save file will contain the entire current GameState including the player's position, the ghost positions, the current maze, the current score and the remaining lives. When loading a saved game, the JSON file will be deserialized to load the saved GameState. These file operations will be handled through IO, keeping game logic separate.

```
saveGame :: FilePath -> GameState -> IO ()
loadGame :: FilePath -> IO (Either String GameState)
```

Custom levels: We will define an ASCII-based plain-text format for custom maze generation. Players will be able to import a .txt map file which will then be parsed into our internal Grid representation.

```
parseGrid :: String -> Grid
loadCustomLevel :: FilePath -> IO Grid
```

2.6 Abstraction and Typeclasses

Since both PacMan and Ghost have a position and a direction on the grid, we want to avoid writing duplicate functions for moving and updating them. To solve this, we will create a Movable typeclass that both data types implement. Alongside our own typeclass, we also use standard typeclasses like Eq and Show, as well as ToJSON and FromJSON from the Aeson library for saving the game. Finally, when updating or rendering the Grid, we will use higher-order functions like map, filter and foldr instead of writing out manual recursion everywhere.

```
class Movable a where
  getPos :: a -> Position
  getDir :: a -> Direction
  setPos :: Position -> a -> a
```

```
instance Movable PacMan where
  getPos = pacmanPos
  getDir = pacmanDir
  setPos pos p = p { pacmanPos = pos }

instance Movable Ghost where
  getPos = ghostPos
  getDir = ghostDir
  setPos pos g = g { ghostPos = pos }
```